PREPROCESSOR

Introduction to Preprocessor Directives 2
Features of Preprocessor Directives 2
Types of Preprocessor Directives 2
1. Macro Expansion Directives 3
A) Object-like Macros 3
B) Function-like Macros 4
C) Nested Macros 5
D) Multi-line Macros 6
E) Special Predefined Macros 7
Difference Between Macro and Function in C 8
Special Meaning of # Inside Macros 9
Syntax 11
Example 1: Joining Variables 11
Expansion 11
Output 11
Example 2: Function Name Creation 11
Expansion 12
Output 12
Uses of ## 12
2. File Inclusion Directives 13
Types of File Inclusion 14
A) Standard File Inclusion 14
B) User-defined File Inclusion 14
3. Conditional Compilation Directives 15
A) #ifdef 17
B) #ifndef 17
C) #if 18
D) #else 18
E) #elif 19
F) #undef 20
4. Miscellaneous Preprocessor Directives 21
A) #pragma 21
B) #error 21
C) #line 22
D) #include_next 22
Practice Questions on Preprocessor Directives and Macros 24
Part A: Theoretical Questions 24
Part B: Multiple Choice Questions (MCQs) 24

Introduction to Preprocessor Directives

Preprocessor directives are special instructions in the C language that are processed before the actual compilation starts.

The C Preprocessor is a program that works before the compiler.

It performs tasks like:

- Including header files
- Replacing macros
- Conditional compilation
- Compiler control

All preprocessor directives begin with the symbol:

```
#
```

Example:

```
#include <stdio.h>

#define PI 3.14
```

Here:

- `#include` tells the preprocessor to include a file.
- `#define` tells the preprocessor to create a macro.

Features of Preprocessor Directives

- Executed before compilation
- Do not end with semicolon `;`
- Start with `#`
- Used to make programs modular and efficient and readable
- Help in debugging and portability

Types of Preprocessor Directives

Preprocessor directives are mainly divided into four categories:

1. Macro Expansion
2. File Inclusion
3. Conditional Compilation
4. Miscellaneous Directives

1. Macro Expansion Directives

Macros are used to replace text automatically before compilation.

The directive used is:

```
#define
```

A) Object-like Macros

These macros replace constants or values.

Syntax

```
#define macro_name macro_expansion
```

Example

```
#include <stdio.h>          // Includes standard input/output functions

#define PI 3.14             // Defines the value of PI

int main()

{

    float area, r = 5;      // Declares area and radius; radius is initialized to 5

    area = PI * r * r;      // Calculates the area of the circle



    printf("Area = %f", area); // Prints the calculated area
```

```
    return 0;               // Indicates successful program termination
}
```

Explanation

Before compilation:

```
area = PI * r * r;
```

becomes:

```
area = 3.14 * r * r;
```

Advantages

- Easy to modify constants
- Improves readability
- Reduces repeated typing

B) Function-like Macros

Macros can also behave like functions.

Syntax

```
#define macro_name(arguments) expression
```

Example

```
#include <stdio.h>

#define SQUARE(x) ((x) * (x))
int main()
{
   int a = 5;
   printf("%d", SQUARE(a));
   return 0;
```

```
}
```

Expansion

```
SQUARE(a)
```

becomes:

```
((a) * (a))
```

Why Double Parentheses?

To avoid precedence problems.

Wrong macro:

```
#define SQUARE(x) x*x
```

If used as:

```
SQUARE(2+3)
```

Expansion:

```
2+3*2+3
```

Result becomes wrong.

Correct macro:

```
#define SQUARE(x) ((x)*(x))
```

C) Nested Macros

A macro can use another macro.

Example

```
#include <stdio.h>
```

```c
#define A 10
#define B A+5

int main()
{
   printf("%d", B);
   return 0;
}
```

Expansion:

```
A+5
```

becomes:

```
10+5
```

Output:

```
15
```

D) Multi-line Macros

Used for multiple statements.

Syntax

Use `\` at the end of each line.

Example

```c
#include <stdio.h>

#define DISPLAY \
printf("Hello\n"); \
printf("Welcome\n");
int main()
{
   DISPLAY
```

```
  return 0;
}
```

E) Special Predefined Macros

C provides some predefined macros.

Macro	Meaning
`__DATE__`	Current date
`__TIME__`	Current time
`__FILE__`	Current file name
`__LINE__`	Current line number

Example

```c
#include <stdio.h>          // Includes standard input/output functions

int main()

{

    printf("%s\n", __DATE__);  // Prints the date when the program was compiled

    printf("%s\n", __TIME__);  // Prints the time when the program was compiled

    printf("%s\n", __FILE__);  // Prints the name of the source file

    printf("%d\n", __LINE__);  // Prints the current line number
```

```
    return 0;                   // Indicates successful program
termination

}
```

Difference Between Macro and Function in C

Feature	Macro	Function
Definition	Defined using #define	Defined using normal function syntax
Processed By	Preprocessor	Compiler
Execution	Code replacement before compilation	Function call during execution
Memory	No separate memory allocation	Separate memory created
Speed	Faster (no function call overhead)	Slightly slower
Type Checking	No type checking	Type checking available
Debugging	Difficult	Easier
Recursion	Not possible	Possible
Return Type	No return type	Has return type
Argument Evaluation	Arguments may evaluate multiple times	Arguments evaluated once

Semicolon	No semicolon after definition	Function body uses normal syntax

Special Meaning of # Inside Macros

Inside a macro, # is called the:

Stringizing Operator

It converts a macro argument into a string.

Syntax

```
#define macro(argument) #argument
```

Example 1: Simple Stringizing

```c
#include <stdio.h>

#define DISPLAY(x) printf(#x)

int main()
{
   DISPLAY(Hello World);

   return 0;
}
```

Expansion

```
DISPLAY(Hello World)
```

becomes:

```
printf("Hello World")
```

Output

```
Hello World
```

Example 2: Printing Variable Name

```c
#include <stdio.h>

#define SHOW(x) printf(#x " = %d", x)

int main()
{
   int a = 10;

   SHOW(a);

   return 0;
}
```

Expansion

```c
printf("a" " = %d", a)
```

String literals combine automatically:

```c
"a = %d"
```

Output

```
a = 10
```

Uses of # Operator

- Debugging macros
- Logging
- Printing variable names
- Creating messages dynamically

Important Note

`#` works only inside macro definitions.

2. ## Operator

The ## operator is called the:

Token Pasting Operator

It joins two tokens into one token.

Syntax

```
#define macro(a,b) a##b
```

Example 1: Joining Variables

```c
#include <stdio.h>

#define MERGE(a,b) a##b

int main()
{
   int num1 = 100;

   printf("%d", MERGE(num,1));

   return 0;
}
```

Expansion

```
MERGE(num,1)
```

becomes:

```
num1
```

Output

```
100
```

Example 2: Function Name Creation

```c
#include <stdio.h>

#define FUNC(name) display_##name()

void display_data()

{

   printf("Data Function");

}

int main(){

   FUNC(data);

   return 0;

}
```

Expansion

```c
display_##data()
```

becomes:

```c
display_data()
```

Output

```
Data Function
```

Uses of ##

- Dynamic variable names
- Function generation
- Generic programming
- Embedded systems macros
- Register manipulation macros

Difference Between # and

Operator	Purpose
#	Converts argument into string
##	Joins two tokens

Example Comparison

```
#define STR(x) #x

#define MERGE(a,b) a##b
```

STR Example

```
STR(Hello)
```

becomes:

```
"Hello"
```

MERGE Example

```
MERGE(num,1)
```

becomes:

```
num1
```

2. File Inclusion Directives

Used to include contents of another file into the program.

Directive used:

```
#include
```

Types of File Inclusion

A) Standard File Inclusion

Used for standard header files.

Syntax

```
#include <filename>
```

Example

```
#include <stdio.h>
```

Meaning compiler searches the file in standard system directories.

B) User-defined File Inclusion

Used for user-created header files.

Syntax

```
#include "filename"
```

Example

```
#include "myheader.h"
```

Meaning the compiler first searches the current folder.

Example of User-defined Header File

File: add.h

```
int add(int a, int b)
{
  return a+b;
}
```

Main Program

```
#include <stdio.h>
#include "add.h"

int main()
{
  printf("%d", add(2,3));

  return 0;
}
```

Advantages of File Inclusion

- Code reuse
- Modular programming
- Easier maintenance
- Reduces code duplication

3. Conditional Compilation Directives

Conditional compilation is a feature in programming that allows you to include or exclude parts of code **at compile time**, based on certain conditions (like platform, debug mode, compiler flags, etc.).

Instead of deciding behavior while the program runs, conditional compilation decides **what code even gets compiled**.

Why use conditional compilation?

- Support multiple platforms (Windows, Linux, embedded systems)
- Enable/disable debug code
- Optimize performance (remove unnecessary checks)
- Handle compiler or hardware differences

How it works (general idea)

You define conditions using **preprocessor directives** or compiler features, and the compiler includes only the code that matches.

Preprocessor Directives Used

Directive	Meaning
#if	Checks condition
#ifdef	If macro is defined
#ifndef	If macro is NOT defined
#else	Alternative block
#elif	Else-if condition
#endif	Ends conditional block
#define	Defines macro
#undef	Removes macro

A) #ifdef

Means:

```
If macro is defined
```

Syntax

```
#ifdef macro

statements

#endif
```

Example

```c
#include <stdio.h>
#define TEST
int main(){
#ifdef TEST
   printf("Macro defined");
#endif
return 0;
}
```

B) #ifndef

Means:

```
If macro is NOT defined
```

Syntax

```
#ifndef macro

statements
```

```
#endif
```

Example

```
#include <stdio.h>
int main()
{
#ifndef TEST
   printf("TEST not defined");
#endiF
   return 0;
}
```

C) #if

Checks constant expressions.

Syntax

```
#if condition

statements

#endif
```

Example

```
#include <stdio.h>

#define VALUE 10

int main()
{
#if VALUE > 5
   printf("Greater");
#endif

   return 0;
}
```

D) #else

Works like normal else.

Example

```
#include <stdio.h>

#define A 2

int main()
{
#if A > 5
   printf("Big");
#else
   printf("Small");
#endif

   return 0;
}
```

E) #elif

Else-if condition.

Example

```
#include <stdio.h>

#define NUM 2

int main()
{
#if NUM == 1
   printf("One");

#elif NUM == 2
   printf("Two");

#else
   printf("Other");
#endif

   return 0;
```

```
}
```

F) #undef

Used to remove a macro definition.

Syntax

```
#undef macro_name
```

Example

```
#include <stdio.h>

#define A 10

#undef A

int main()
{
#ifdef A
   printf("Defined");
#else
   printf("Undefined");
#endif

   return 0;
}
```

Header Guards

Used to prevent multiple inclusion of same header file.

Example

```
#ifndef MYHEADER_H
#define MYHEADER_H

// header contents

#endif
```

Why Header Guards?

Without guards:

- Same file may include multiple times
- Causes redefinition errors

Header guards solve this problem.

4. Miscellaneous Preprocessor Directives

These directives provide additional compiler control.

A) #pragma

Used to give special instructions to the compiler. Compiler-specific directive.

Syntax

```
#pragma instruction
```

Example

```
#pragma warning(disable : 4996)
```

Uses of #pragma

- Disable warnings
- Control structure packing
- Compiler optimization

B) #error

Generates a user-defined compile-time error.

Syntax

```
#error message
```

Example

```
#ifndef VERSION

#error VERSION not defined

#endif
```

C) #line

Changes line number and filename for compiler.

Syntax

```
#line number "filename"
```

Example

```
#line 100 "test.c"
```

After this, compiler treats next line as line 100.

D) #include_next

Used in advanced programming to include next matching header file. Mostly used in system programming.

Important Notes About Preprocessor

Directive	Purpose
`#define`	Create macros
`#include`	Include files
`#if`	Conditional compilation
`#ifdef`	Check if macro defined

#ifndef	Check if macro not defined
#undef	Remove macro
#pragma	Compiler instruction
#error	Generate error

Advantages of Preprocessor Directives

1. Code reusability
2. Easier debugging
3. Better portability
4. Faster development
5. Simplifies large projects

Disadvantages

1. Macros do not perform type checking
2. Debugging macros may be difficult
3. Incorrect macros can create hidden errors

Conclusion

Preprocessor directives are an important feature of the C language. They help perform tasks before compilation such as:

- Macro replacement
- File inclusion
- Conditional compilation
- Compiler control

They improve:

- Efficiency
- Modularity
- Maintainability

Understanding preprocessor directives is essential for writing professional C programs.

Practice Questions on Preprocessor Directives and Macros

Part A: Theoretical Questions

1. What is a preprocessor directive in C? Explain its role.

2. What is a macro? Explain object-like and function-like macros with examples.

3. Differentiate between macro and function.

4. Explain file inclusion directives in C.

5. What is conditional compilation? Explain #ifdef and #ifndef.

6. Explain the # and ## operators with examples.

7. What are predefined macros in C? Explain any four.

8. What is #pragma directive? Explain its applications.

9. What is #error directive? Explain with suitable example.

10. Explain header guards and their importance.

Part B: Multiple Choice Questions (MCQs)

11. Which symbol is used to start a preprocessor directive?

A) $

B) @

C) #

D) %

---

12. Which directive is used to define a macro?

A) #macro

B) #define

C) #include

D) #ifdef

---

13. Which directive includes header files?

A) #include

B) #define

C) #pragma

D) #ifdef

---

14. Which directive removes macro definition?

A) #remove

B) #clear

C) #undef

D) #delete

---

15. Which operator converts macro argument into string?

A) ##

B) #

C) &&

D) &

---

16. Which operator joins two tokens?

A) #

B) ##

C) %

D) ::

---

17. Which directive is used for conditional compilation?

A) #if

B) #include

C) #pragma

D) #error

---

18. Which directive generates compile-time error?

A) #warning

B) #pragma

C) #error

D) #define

---

19. Which directive gives compiler-specific instructions?

A) #ifdef

B) #pragma

C) #include

D) #line

---

20. Which directive checks whether macro is defined?

A) #ifndef

B) #ifdef

C) #endif

D) #undef

---

21. Which directive checks whether macro is NOT defined?

A) #ifdef

B) #ifndef

C) #endif

D) #pragma

---

22. What is the output?

```
#define A 5+5

printf("%d", A*2);
```

A) 20

B) 15

C) 10

D) Error

---

23. What is the output?

```
#define SQUARE(x) x*x


printf("%d", SQUARE(2+3));
```

A) 25

B) 11

C) 10

D) Error

---

24. Which predefined macro gives current line number?

A) __TIME__

B) __FILE__

C) __LINE__

D) __DATE__

---

25. Which predefined macro gives current date?

A) __DATE__

B) __TIME__

C) __FILE__

D) __LINE__